

CX Intelligence Suite - Package and Technology Inventory

Python packages, frontend technologies, model components, and approximate contribution percentages.

Cx Intelligence Suite - Package And Technology Inventory

Prepared for IT review

Version scope: IntelligenceSuite-WebVersion and equivalent desktop packaging approach

Last updated: 2026-07-01

1. APPLICATION SUMMARY

Tool name:

Kestrel Analysis Suite / CX Intelligence Suite

Primary purpose:

Analyze CSAT and NPS survey workbooks, generate performance dashboards, statistical results, role-based insight lenses, sentiment output, theme classification, ACPT classification, resolution status, and export-ready outputs.

Deployment options:

1. Web version hosted as a Python web service.

2. Desktop version packaged as a standalone local app with bundled runtime/dependencies.

2. RUNTIME STACK

Web runtime:

Python 3.11.9

Web server:

Python standard library HTTP server pattern using ThreadingHTTPServer/BaseHTTPRequestHandler in backend/server.py

Frontend:

Static HTML, CSS, and JavaScript.

No Node.js build step is required for the current WebVersion.

No package.json is present in the WebVersion.

Render deployment:

Build command:

pip install --upgrade pip && pip install -r requirements.txt

Start command:

python backend/server.py

Health check:

/healthz

3. PYTHON PACKAGES FROM requirements.txt

pandas==2.2.2

Purpose: workbook/CSV data processing, tabular analysis, column profiling, export preparation.

numpy==1.26.4

Purpose: numeric operations, arrays, supporting statistical calculations.

openpyxl==3.1.5

Purpose: Excel .xlsx reading/writing through pandas and direct workbook handling.

python-pptx==0.6.23

Purpose: PowerPoint/report generation support.

scikit-learn==1.5.1

Purpose: Owl model training and classification pipeline, logistic regression, train/test split, metrics.

joblib==1.4.2

Purpose: saving/loading trained Owl model artifacts.

sentence-transformers==3.0.1

Purpose: embedding model support for Owl Theme/ACPT/Resolution classification.

transformers==4.44.2

Purpose: Sparrow sentiment model support.

torch==2.3.1

Purpose: transformer model runtime.

safetensors==0.4.3

Purpose: safe model tensor file loading for transformer/sentence-transformer models.

scipy==1.13.1

Purpose: statistical/numeric support used by scikit-learn and analysis routines.

4. PYTHON STANDARD LIBRARY MODULES USED

The app also uses Python standard library modules. These do not need separate pip installation.

Observed examples:

argparse
base64
cgi
collections
dataclasses
datetime
functools
hashlib
hmac
http.server
importlib.util
io
json
math
mimetypes
os
pathlib
pickle
platform
random
re
secrets
shutil
socket
subprocess
sys
tempfile
threading
time
traceback
typing
urllib
uuid
webbrowser

5. JAVASCRIPT / FRONTEND TECHNOLOGY

JavaScript:

Vanilla JavaScript only.

No React, Angular, Vue, Next.js, Vite, or Webpack build chain is used in the tracked WebVersion.

CSS:

Plain CSS files.

HTML:

Static HTML entry pages served by the Python backend.

Browser features used:

Fetch API for backend calls.

DOM event handling for guided workflows and dashboards.

Local UI rendering through client-side JavaScript.

Dialog/pop-up components through HTML/CSS/JavaScript.

Window open/navigation for training modules and dashboards.

6. ML / NLP COMPONENTS

Sparrow:

Purpose: sentiment classification from verbatim comments.

Model family: transformer-based local model.

Runtime packages: transformers, torch, safetensors.

Bundled model folder observed:

models/sparrow_cnx_sentimentmodel/

Owl:

Purpose: Theme, ACPT, and Resolution Status classification.

Model family: sentence-transformer embeddings plus trained scikit-learn classifier.

Runtime packages: sentence-transformers, scikit-learn, joblib, torch, safetensors.

Bundled model folder observed:

models/theme_acpt_resolution_model/

Important note:

For web deployment, large model files should normally be handled through persistent disk or external storage rather than repeatedly committing new trained artifacts to Git.

7. APPROXIMATE TECHNOLOGY CONTRIBUTION

Method:

Contribution is calculated by tracked file size in the Git repository. Two views are provided because model files are much larger than application source code.

7.1 Application source contribution

Scope:

Tracked app source files only: Python, JavaScript, CSS, HTML, JSON/config, YAML.

Excluded:

Model artifacts, images, PDFs, sample data, and UAT documentation.

Approximate contribution:

JavaScript: 61.93%

Css: 19.57%

Python: 10.85%

Html: 6.90%

JSON / Config: 0.74%

Yaml: 0.01%

Interpretation:

Most visible application behavior is frontend-driven JavaScript. Python is the backend/API/data-processing layer. CSS and HTML carry the UI presentation.

7.2 Full repository size contribution

Scope:

All tracked files including model artifacts, documentation, images, PDFs, and sample files.

Approximate contribution:

Model artifacts: 95.54%

Pdf: 1.65%

Images: 1.21%

Json: 0.70%

JavaScript: 0.53%

Css: 0.17%

Python: 0.09%

Html: 0.06%

Excel: 0.03%

Markdown: 0.02%

Csv: 0.01%

Text/YAML/metadata: less than 0.01%

Interpretation:

Full repository size is dominated by bundled ML model files. This does not mean the application is mostly model code; it means binary model artifacts are large.

8. MAIN APPLICATION MODULES

frontend/

Suite landing page, authentication helper, app navigation.

backend/server.py

Main Python web server and API route handler.

backend/model_worker.py

Model worker support.

apps/nps-analyzer/

NPS guided analyzer, NPS analysis engine, review dashboard.

apps/csat-analyzer-next/

CSAT guided analyzer and review dashboard.

apps/nps-intelligence-hub/

NPS detailed dashboard/intelligence hub.

apps/csat-intelligence-hub/

CSAT detailed dashboard/intelligence hub.

apps/sparrow-training/

Sentiment model training workspace.

apps/theme-model-training/

Owl Theme/ACPT/Resolution Status model training workspace.

models/

Bundled model artifacts and model metadata.

outputs/

Generated outputs.

temp/

Temporary processing files.

logs/

Runtime logs.

9. DESKTOP VERSION DEPENDENCY POSITION

For the desktop version, the same Python package set should be bundled with the application or included in the packaged runtime. Users should not need to install Python manually if the desktop package includes:

- 1. Python runtime.**
- 2. Required packages from requirements.txt.**
- 3. Static frontend assets.**
- 4. Backend Python server.**
- 5. Required model artifacts.**
- 6. Launcher EXE.**

10. WEB VERSION DEPENDENCY POSITION

For the web version, Render or another hosting platform installs Python packages from requirements.txt during deployment. The hosted service then serves frontend files and backend APIs from the same application.

Recommended model storage approach for web:

- 1. Keep source code in Git.**
- 2. Avoid committing newly trained large model files unless approved by IT.**
- 3. Use persistent disk or external storage for trained model artifacts.**